INSTRUCTION MANUAL

SPECTRAVIDEO™

TUTORIAL PROGRAM

INTRODUCTION TO BASIC™

FOR THE SPECTRAVIDEO

SV·318/SV·328

PERSONAL COMPUTER SYSTEM

TAPE

INTRODUCTION TO BASIC

INTRODUCTION

Welcome. This tutorial cassette is designed to elevate your programming skill to new heights. You will encounter some programs that you have seen before and many that you have not. Throughout this tutorial we will mention dozens and dozens of programming tips that professional programmers use. And why shouldn't we mention them to you. After all, our goal is to show you how to take full advantage of the unique features of your SV-318 computer.

Published by
SPECTRAVIDEO INTERNATIONAL LTD.

First Edition
First Printing 1983
Printed in Hong Kong

SD · 229T · UM

Our first program introduces two basic programming concepts. One is the use of a container to hold a given value. And, two, is the use of a loop to calculate a single set of instructions a given number of times.

Consider the following: Your relative gives you a penny as a gift on your first birthday. He promises to double the amount of the gift each year until you reach your twenty first (21st) birthday. How many pennies will you receive from him on your twenty first birthday?
The initial value of the gift is one cent, you must begin your program by initializing a container, let's call it "T", to hold a value of one. "Initializing" is the computerize term that means to assign an initial value to something.

This amount will double. Therefore "T" wil equal "T" plus "T", each year until your twenty first birthday. That is, "T" will double twenty times in all. Rather than write twenty statements to that effect, you can simply write a FOR-NEXT loop.

CLOAD and LIST the first program

Into 0 -2 7

Line 10 initializes container "T" at one. Line 20 begins our loop. When the computer first encounters this line, it sets the value of "X" at 1. It then proceeds to line 30, doubling the value of "T". Line 40 simply prints the value of "T" at a given point in the loop. The crucial line is line 50 which contains the instrucion, NEXT X. The computer, seeing this line, increases "X" by one, and returns to line 20. It then compares the value of "X" with the bounds set in line 20, which in our example, are numbers one through twenty. As long as the value of "X" does not exceed these bounds it will execute the loop. When however, "X" reaches the value twenty one it fails the test and drops through to the next statement.
Toi repeat, our program will pass through the loop twenty times and print the value of the gift on each of your twenty birthdays (ages two thru twenty one).
Type RUN and see for yourself.

Please note: We set the boundaries of our loop at one to twenty. It would have bee equally correct and perhaps simpler to visualize, had we set these to two through twenty one. At each birthday, second, third, etc., through your twenty first, the gift will be doubled, that is,

"T" equals "T" plus "T". Either way you will be doubling your initial value of one cent twenty times in all.

Better yet, let's say you wished to set the bounds at one to twenty one. The computer would then execute the loop twenty one times in all. What do you need to change in order to make this work out? Simply change the initial value of "T" to one half so that the value of the gift on your first birthday is one half of one cent, and you're in business.

CLOAD and RUN the revised version and you'll see what we mean. The computer will execute three different routines and yield the same results each time. Remember. There is no one right way to program. You just strive to write the best, most logical and efficient program you can. After you RUN the program, type LIST

O.K. You've just printed the value of the gift on each succeeding year. Did you notice that this time the results were printed in two columns? That is because the print statement included a comma. That tells the computer to print "T" in the next column instead of automatically returning you to the next line. Now let's have the computer print the total amount you will have gotten through the years in pennies and dollars.

CLOAD and LIST the updated program,

Note the three new lines. Line 5 introduces another container to help keep track of your finances. It is initialized at one, just like "T" in line 10. Line 35 sums the new value "T" with what you already have in the bank. The new balance is called "Q" and equals the old "Q" plus "T". Line 70 prints the total number of pennies received. That is the value of "Q" at the end of the program. In addition, it cfore. You're used to calculate the number of whole dollars received. The integer function, INT, "Q", divided by 100 then disregards the numbers to the right of the decimal point. For example, 128 divided by 100 equals 1 point 28. The INT of 1 point 28 divided by 100 would simply be one. Now run the program and see how it works.

You're in luck. Your relative has decided to change the initial gift, from one penny to one dime. And get this, he may change his mind again and start with a dollar instead. Change the program to calculate a dime and dollar gift. Don't worry. Most of the program stands as is. Simply change the initial values of "T" and "Q".

CLOAD, LIST and RUN, and enjoy your newly found riches.

Now that you are used to the FOR-NEXT statement, let's introduce what we call nested loops and make things even more exciting.

CLOAD and LIST the next program. Clock 24

This program is the clock project. Line ten, twenty and thirty set the bounds for the hours, minutes and seconds in a day. H equals 0 TO 23, M equals 0 to 59. and S equals 0 to 59. Think about it. If time started right now, at zero hour, zero minute and zero second, you would need to proceed from zero, zero, zero to 23:59:59 to complete a one day cycle.

Let's move on. Line 40 clears the screen. Line 50 gives a printout of the exact time and line 60 sounds off. Line 70 merely provides a time delay. It is a simple FOR-NEXT loop allowing you time to see what is going on. A simple technique, but one that is very useful to keep the computer under your control. 483 = 1 sec

And now, for the serious stuff. Remember, we are dealing with nested loops. Line 80 increases "S" and returns you to line 30. It will do this until "S" reaches the value of 60. At that crucial moment, the program will fall through to line 90 where "M" will be increased and return control to line 20 and begin counting seconds all over again. Of course once "M" exceeds its given bounds line 90 will defer to line 100 sending the program back for the hourly update and off it goes again. Amazing isn't it? Instead of a long, tedious, repetitive program, with just three simple loops the computer will do our work for us. yes, three loops. Remember, the extra "T" loop in line 70 simply provides a time delay between printouts on the screen.

Oh! One last thing before you start the clock. Look again carefully at your screen. Notice the structure of our loops. Each NEXT statement refers back to the most recent FOR statement. In BASIC you are not allowed to intertwine your loops. If you first used a FOR A statement and then a FOR B statement, then the NEXT B statement must precede the NEXT A statement.

O.K. RUN the program, sit back and watch time tick away. LIST the program

Look again at your screen. To display a given time of day, you need three containers, one for seconds, one for minutes and yet another for hours. Always remember to label the important elements in your program. Note: Labelling is just another way of saying: assign a container. In our case we assigned "H" for hours, "M" for minutes and "S" for seconds. Labels that make sense are easy to understand later on.

Now, consider our case. To make time tick, you need to increase the seconds container one second at a time. But watch out! When your second hand reaches sixty, what do you want to see happen? What else but your minute hand move up one and your second hand start over at zero. The same situation occurs at the end of each hour. Just

such a case cries out for nested loops, a quick and easy way to repeat procedures the 'just enough but not too often' number of times and without getting your hands dirty.

How about adding color and animation to our repertoire of computing skills. Think about it. Animation is merely an illusion. Put an object in one spot, erase it and put it in an adjacent spot and you'll have all your friends believing it moved! Add some color, and things really start brightening up.

Suppose, for example, you wanted to create the illusion of a ball tumbling down stairs. How would you do it? First, build yourself a staircase. Use points instead of bricks, each at consecutive positions. That is, one over and down, one over and down, etc. Of course, you only need to set up a loop and the computer will do all the heavy moving for you.

CLOAD and LIST and see what we mean.

Bounce ~~Code 25~~ 27

First, take note of line 10. Your SV-318 has two graphic screens. SCREEN 1 with high resolution and SCREEN 2 with low resolution. For our purposes SCREEN + is like drawing with a crayon instead of a fine point pen and serves us just fine. Always remember to include graphics commands in a program to tell the computer you want to draw pictures.

Line 20 initializes a container called "Y" to zero. In fact, had we left this line out, the computer would have done it for us, because all values are zero when first encountered. Line 30 assigns "X" the values between O and 190. Line 40 should be very familiar to you by now. It was one of the first commands we introduced you to in the manual. It picks a point and lights it up in color number three which is green. Line 50 increases the value of "Y" the vertical coordinate, which together with the increase to coordinate "X" given in line 60 creates the impression of a descending staircase. It's that simple.

Fine, we've drawn a staircase, descending from position 0 comma 0 which is the top left corner of the screen to position 190 comma 190. But where is the ball? Go back to the top of the screen by resetting the "Y" value to zero, that is, zero positions down. Now you want the ball to start on the top step and work its way down. Confused? It's easy if you take one step at a time.

Lines 90 through 100 mirror the effect of lines 30 through 40, but with two important differences. Each point is five positions over to the right, giving the impression of a seperate object rippling down off the stairs. The change in color suggests a seperate object to the viewer as well. The real trick here, however, is the suggestion of movment, that

the ball is here one minute and there the next. That's it: set it here and then erase it setting it one position away from its previous position. Turning the points on and off is this fashion gives you the effect you're looking for. The time delay in lines 110 and 120 allows you to perceive the switch of positions. Line 160D sends you back to the top of the staircase with just enough time to catch your breath.

On your mark... RUN the staircase.

Now that you you've learned a few fancy tricks, let's go back and learn some basics. In the birthday gift program you learned about the integer function. Here we will introduce a new concept, that of R-N-D, or random number. In effect, it returns a random number between point 0000 and point 9999.

CLOAD and LIST IT and we'll take a closer look

Random 30

You may have noticed in our previous program a line statement beginning with the word REM. This is the abbreviation for remark. The computer overlooks it when executing the program. It does however list it for you allowing you to interject remarks along the way. It is a painless way to keep track of what is going on and makes your program easier for a newcomer to understand.

Our remarks introudce you to the number guess game. The computer chooses a number from one to one hundred at random. The object of the game is to guess the chosen number in as few guesses as possible. How will the computer choose a random number? Look at line 20.

As we said earlier, RND chooses a number between point 0000 and point 0000. We're looking for a number between one and one hundred. Not to worry. Simply multiply the number chosen by RND by one hundred and take the integer value of the result with the INT command. For example. In a given turn the RND function returns the value point 468. Multiplying by 100 gives you 46 point 8 the integer part of the value is simply 46, which is a number between one and one hundred. Great but look again. Line 20 also adds one to the resulting nuber. Why is this necessary? Consider the following: Suppose the RND function returns the number point 009. Multiplying by 100 results in 0 point 9. If we were take the integer part of this number we will get a zero which fails our requirement that the chosen number be between one and one hundred. To insure that this does not occur, simply add one to your result.

The command minus TIME in line 20 insures that we receive a truly random number each time the program is run. Without the minus TIME

instruction the seed number, or beginning number the computer starts with is always the same when RND is used. This is called a pseudo random number. Minus TIME makes it a truly random seed number.

Fine. Go ahead and look at line 30. Container "Y" on line 30 is being used as a counter. Initialized at zero, "Y" is our way to count the number of guesses that you make. It will be increased by one each time you enter your guess by the instructions on line 100. As you already know, the command LET is one way to assign a value to a container. It is optional. "Y equals Y plus 1" will have the same effect. LET merely sounds friendlier.

O.K. Back to line 40. The next few lines are self explanatory. They merely print a message to the viewer. Wait... there is one important feature here. Line 90 ends with INPUT X. Hopefully you remember that using the INPUT instruction is one way to allow interaction between you and the computer. INPUT prints a question mark and waits for you to enter a value and then assigns this value to the specified container. The program will not continue until you have your say and press the ENTER key.

RUN the program and see how it works. Then LIST it

We're back again and the program should be on the screen. Look again at line 110. On your first guess "Y" will equal one, by adding one to the initial value of zero. Then if your choice which is stored in "X" equals the number chosen by the computer which is stored in container "Y", the computer will pass control to line 140 telling you of your success and asking you if you wish to play again.

Let's suppose however that your choice exceeds the random number. The test in 110 will fail and then moves on to line 120 which then passes control to line 210. Upon arriving at line 210 you will be told that your guess was too high and that you must try again and so you are returned to line 90.

One other thing though. Look again at line 140. If your guess neither equals the random number chosen by the computer that is stored in container "R" nor exceeds it, the computer passes through lines 110 and 120 reaching line 130 which sends you to the message on line 230 that informs you that your guess was too low. It is important to note here that there is no need to explicitely test to see if your guess is less than the number stored in "R". Because line 130 will not be read by the computer if your guess, which is stored in "X" is less than the number stored in "R". If both the tests on line 110 and 120 prove to be false there is no other possibility than "X" is less than "R" and since ther is no other way to reach line 130, if it is indeed reached, then the computer knows that "X" is less than "R" and continues accordingly.

Moving right along. Let's try and fire at a moving dot. By now you already know that the effect of movement is accomplished by setting and resetting points. With this in mind, try to picture a cannon, mounted in a fixed position, aimed at a moving target. The cannon's aim is not adjustable. When you fire the target needs to be in perfect range, otherwise, the enemy will win. What do you do first?

CLOAD and LIST It.

Cannon 35

Line 10, as you know is a remark. When and key is pressed a dot is fired at the moving dot. If they collide, they will explode and the game will restart.

The remarks on line sten thru sixteen tell you what the program is about. The screen two command on line 20 is the command to set the computer to draw on the low resolution graphics screen. Line 30 places the cannon on the screen. Do you remember what each of the three numbers mean? The first pair of numbers in the parentheses are the coordinates of the point we which to light up. That is, sixty points over from the top left corner of the screen and 96 points down. Three is the color number being used. Now the cannon is positioned. Now for the target.

Lines 40, 50, 90 and 100 cause a dot, which will serve as the target, to move from the top of the screen to the bottom of the screen. But wait, what are lines 60, 70 and 80 doing in the middle of the loop that controls the movement of the target? Line 60 is a standard time delay, the kind we have encountered many times before. Lines 70 and 80 are new to you however. The nstructions on line 70 is the computer's way of keeping tabs on what you are doing. It says that whatever key on the keyboard you press will be stored in the container "A" dollar sign. For example, if you hit the "L" key, then "A" dollar sign will equal "L". This method is similar to the input statement that you have already learned. The differece is that the INPUT command accepts whole words, while the INKEY dollar sign accepts only one key.

Now look at line 80. If "A" dollar sign does not equal the empty set then goto line 200. Be sure that you understand the code on line 80. Theless than, greater than combination of symbol implies inequality. The quote marks symbolize the absence of any key. Thus we have: if "A" dollar sign equals anything, in other words, if any key is pressed, then goto line two hundred. On the other hand if no key is pressed the point that was originally set in line 50 is now reset and the target continues to move down the screen.

Notice that line 110 will recycle the target back to the top of the screen in the event that it made it all the way down without your intervention.

O.K. We are ready to see what happens when you fire the cannon by pressing any key and the program jumps to line two hundred. Lines 200,210 240, and 250 creat the same effect as the first loop in the program. A point is switched on and off thereby causing a dot to move from the top to the bottom of the screen. Line 220 performs a very crucial test. It tests to see whether the cannon fire hits its mark. If the point that is fired from the cannon, which travels from left to right on the screen and was set in line 210, meets up with the point of the target which travels from top to bottom, and was set in line 50, then BOOM, the target will explode. It reads as follows: If point 179 comma 96 which is the cannon's fire is set to color number three and the color of the target is number six then control will pass to line 300.

At that point the screen is cleared and an explosion is heard. The simple command in line 310 adds this exciting feature. This instruction will be explained in a later program on this tape. Line 320 allows for a time delay and then line 330 reroutes the computer back to the beginning of the program for another round.

That's all for our explanation of this program. Go ahead RUN the program and try your luck.

It's time to mvoe from the arcade room to the gambling hall. Our next program will simulate the flipping of a coin. Consider the following. You throw a coin into the air and watch it land. How often will it land on its head and how often will it land on its tail? It's random isn't it? Is that enough of a hint?

CLOAD and LIST the next program cflips 39

Three containers are set up in line 20. "A" and "B" for heads and tails and "T" for the number of coins that are flipped.

Just one other thing. If "X" which is the container that stores the value of the coin flip, passes the test in line 50, then "A" will be increased by one. If line 50 is true then control would pass through line 60 without changing the value stored in container "B". If the test on line 50 would have failed then container "A" would remain unchanged while container "B" was changed because the condition on line 60 would have been true.

How can we avoid the unnecessary testing on line 60? If "X" equals one then we need not test to see if "X" equals two, right? Well, we can add the instruction, GOTO 70 at the end of line 50. Then if "X" was equal to one then "A" would be increased by one and the computer would skip line 60 and go directly to line 70. Why didn't we show you this to begin with? The truth is that program design is always a give a take. Memory space for extra instructions must be considered versus speed of execution. In this case, the time it takes to

run the test on line 60 is neglible. In just such a case, the programmer might decide to leave out the "GOTO 70" command. The "GOTO 70" command would however, be a technically neater construction.

Now RUN the program and try it out.

It's time to take care of a few odds and ended before we bring in some more graphics detail. So far we have assigned values to containers in any one of three ways. The "LET" technique, for example "LET X equal to one", the INKEY dollar sign instruction and the INPUT command. Here we will add the READ-DATA combination. As you know, every FOR statement needs a NEXT statement. Similarly, every READ statement needs a DATA line.

CLOAD and LIST the program and you'll see what we mean,

Grades 62

Line 100 provides the data which will be assigned to container "TG". Each time the computer encounters a READ stetement the next available value in the data line is suplied. The value is not inctrouced at the keyboard by the user, nor is it fixed in a LET command. In this fashion one can include a whole list of data at once and should ever the need arise to change that set of information, a simple retyping of one line that is easily found will do the trick.

But that isn't the only way new item we have included inthis program. Suppose that you wished to organize the grades of students on ten tests. Better yet, an individual student wishes to keep track of his results. In such a case you have a whole list of items, each pertaining to one student. You could of course create ten saperate containers to represent the grade. But how could you tell they they were related to each other ? You couldn't if you used that method but there's a better way.

An ARRAY, that "A", "R", "R", "A", "Y", is similar to a parent container. It is a single parent container into whch many smaller containers are placed. In this larger container all the smaller ones atre linked together. "TG" one, written as; "TG", open parenthesis, one, close parenthesis, is the first in this series called an ARRAY. And if the array were set up to hold ten elements then "TG" ten is the last element in the series. This is an easy and convenient way to label and organize a list of items. The only thing you ned to do before you can use an array is to dimension it. A simple "DIM" command in line 20 lets the computer know how many slots to keep open for the larger parent container. You may set the amount of the reserved locations to your discretion.

Keeping in mind what we just explained about the READ-DATA statements and dimensioned arrys, see if you can follow our program.

As we said line 20 reserves ten small containers each linked to the parent container called "TG". Lines 30 through 50 reada value from the DATA line and place it into each of the smaller containers. So "TG" one is given the first value on the data line and "TG" two is given the second value listed on the data line etc.

Now let's say you wish to know the student's grade on his fourth exam. Line 60 prints the message to propmt you to type in your answer and assigns into a container called "TGZ". Then online 70 the computer prints the "Z" item in the parent container. For example, if you typed in four, the computer will respond with the fourth value listed on the data line because this value is stored in container "TG" four.

Type RUN and see how it works.

Suppose that instead of storing the test results of just one student you wish to store ten test results for two students. In such a case, instead of a single list of ten items, you would have to have use a double array. Think of it as a grid, with the student's name or identification number in the vertical column and his test grades listed in a column next to his name.

Or, suppose that you are in the midst of publishing your first novel. You need to store information in a filing system that will contain the number of chapters in your manuscript together with the number of paragraphs they contain.

Press CLOAD and after the program is loaded LIST it

double L16

Line 15 dimensions an array called "V". Notice that the numbers within the parentheses are five and two. The five is there because there are five chapters in your manuscript, but why is the number two there? Why do you need to arrange two columns for each of your chapters? The answer is simple. One column is used to label the chapter and the second column will be assigned the number of paragraphs it contains. Had you wanted to add some other piece of information about each chapter you would need to add a third column.

Lines twenty through fifty read information from the data lineinto your filing system. "S" begins at one and "I" begins at one. Therefore position "V one comma one" is assigned value one from your data line, and position "V one comma two" is assigned twelve, and position "V two comma one" holds two and position "V two comma two" holds eleven. Get it? Don't worry if you aren't to sure about this stuff right now. As we continue it will become easier. To printout the information that is stored in the double array and retain its two column appearance we have added lines seventy through one hundred.

Before returning to graphics we need to explain one last BASIC concept that involves the manipulation of containers that hold words. In computer talk a container that holds letters and words is called a "string variable". We have worked with this type of container in the tutorial manual. Now we will demonstrate to you how to take string containers apart and put them together in various ways.

CLOAD the next program and then LIST it

String 40

Line 10 reserves enough space in the computer's memory for working with strings. Without it the computer reserves only enough space for a few short words. With the CLEAR instruction you can make the reservation of your choice. Line 20 then requests our input and assigns it to a string container called "S dollar sign. Now look at line thirty. Can you guess what value the L-E-N command gives us? This command counts the number of characters in a string container we tell it to check. In other words it counts the length of the string the container is holding.

The LEFT dollar sign instruction on line 50 starts from the left-most character in the "S" dollar sign container and counts off the number of characters that is stored in the "X" container to the right of the first letter. The RIGHT dollar sign command that is listed on line seventy starts from the right-most character of the "S" dollar sign container and counts off the number of characters that is stored in the "X" container to the left of the last letter. An even more interesting string manipulation that is found on line ninety called MID dollar sign, will return the midsection of the "S" dollar sign container starting at the sixth character from the left side of the word and counting off a total of four characters. It is much easier to understand string manipulation after you have played with it so go ahead and RUN the program and see if we're just stringing you along.

Satisfied? Well, we aren't finished with string manipulation yet. Not only can reprint parts of a given string, leftmost, rightmost or somewhere in between, but using these techniques you can now become copy editor for a leading magazine.

Editor -53 -55

CLOAD and LIST the next exercise and read it through and of course try it out. Remember, the LEN command returns the length of a string container, MID dollar sign returns a piece of a string at the point you determine. When you think you have understood the system RUN it a few times and make sure that it works to your liking.

O.K. Let's get back to graphics. You already know how to set and reset points but that's kidstuff. To do some serious drawing you will need to draw faster than one point at a time. If you wish to draw a line, all you

really should have to tell the computer is the begining point and end point of the line and the computer should fill it in for you. Well that's precisely all you have to do with the LINE command.

CLOAD the next program and RUN it.

Line 57

Now LIST the program and we'll explain it.

Lines 40 and 50 add a touch of randomness to the routine. "X" is a random value between one and 255 and "Y" is a random value between one and 191. Line 60 is the key one here which introudces you to the LINE instruction. This is your way of asking the computer to set the points along the line that begins at the point specified by "X" and "Y" in the first parentheses and ends at the point specified by "Z" and "A" in the second parentheses on line 60. The number six at the end of line sixty specifies the dazzling color to use.

Is this all that we can do with the LINE statement? Not by a mile! A simple addition of the letter "B" will tell the computer to create a box instead of a line. The top left hand corner of the box will have the coordinates of the "X" and "Y" in the first parentheses and the bottom right hand corner will have the coordinates of "Z" and "A" in the second set of parentheses. Here is how to change line sixty.

List the program. Type line sixty again exactly as it appears, and add a comma after the second parentheses and then add the letter "B". Be careful. Line sixty should read: line, open parenthesis, "X", comma, "Y", close parenthesis, hyphen, open parenthesis, "Z", comma, "A", close parenthesis, comma, "B", Oh, and add one more line to clear thescreenotherwise things will get too crowded. Enter 33 CLS.

Now RUN your revised program and box yourself in.

Color can add special effects to the simplest of programs. LIST the program again and type line 60 again. This time add the letter "F" immediately after the letter "B", and RUN it and watch what happens.

See, it's easy. A simple command changes a point into a line, another a line into a box and yet a third one fills that box with color.

By now you have noticed what a difference your choice of screens can make. If you add the right colors and print exactly, and we maan exactly where you want to, cute things can happen. Take a look for yourself.

CLOAD the next program and RUN it. Then type LIST

Locate 61

Remember one thing. You are at the controls. Set your foreground and background colors with the COLOR instruction on line five and then locate your cursor wherever you wish with the LOCATE command and away we go.

Enough playing around. Get your feet back on the floor and let's go inside.

CLOAD and LIST our next program. Take a guess at what it will draw and see how well you've done, by RUNning it.

house 64

Had we asked you to draw a house you might very well have declined. Don't think of it as a house. Think of it as a few lines and boxes thrown together. If you are careful when planning your lines and boxes at the drawing board, then you'll go away with a lot more than random lines.

Relax a minute. Maybe we can pick up a yo-yo and CLOAD and RUN the next program.

Yo Yo 70

Getting dizzy?

LIST the program. No need to type MOTOR ON. I'll wait a few seconds before I continue. If you don't like the colors of the screen, type COLOR 15 comma four to return to the normal screen colors.

First a quick review. The line statement on line seventy draws a line beginning at point 128 comma 0 and ending at the point specified by the two numbers in the second parentheses. Notice that the value of the "Y" container changes as "P" changes from one to eight. It is a simple line statement.

O.K. Now, for a few new concepts. Line 80 adds a new feature, the CIRCLE comand. If you wanted to draw a circle on a blackboard, what things would you need to know? Right, the center point. The numbers within the parentheses on line 80 fix the center point of the circle. Now what is the only other thing absolutely necessary to create a circle? The radious, of course. The number 15 which follows the parentheses on line 80 lets the computer know that you want a circle with a radious of 15 units. That's all there is to it.

Line 30 causes the yo-yo to move downward and line 40 causes it to move up again. The whole process of the down and up motion will only be performed four times because lines 20 and 50 make sure this whole process is repeated only four times. Fun, isn't it?

That circle in your yo-yo needn't be so casual. It's your privielege to add a color code after the radius. Not only that, say you only want half

a circle, a quarter, or an eighth. Set two more numbers to indicate starting points and end points and you've got it. P1, pronounced pie, or 3.14 equals 3 O'Clock and 6.28 equals 9 O'Clock. Sounds like we're going in circles doesn't it. Don't worry. Take a peek at our next program.

CLOAD and RUN it and you'll see it's as easy as 1,2,3. Don't forget to type MOTOR ON.

Circle 72

Hey, wait a minute! Where did that border come from. And that strange "U" shape in the middle? What's going on? Give us a second and we'll walk you thru the progam, one line at a time.

LIST the program. You've probably answered your own questions by now. The COLOR command on line ten not only sets the foreground and background colors but also the border color too. You have options on all three when you're in a graphics screen.

Now look at the REM statement. "C" is set at one to fifteen. That accounts for the flashing colors. Our start and end values are swapped each time we go through the loop. That's the jump rope. Last but not least, we substituted the number two for the normal one one height-width ratio. That's way our one half circle became a "U". Feel free to play around with any or all of these options at your leisure.

All right? You've learned to go from points to lines, and on to boxes and ellipses, the cigar shaped circle. To add reality to your portrait howver, you'll need to add the colors of life. What better way to paint your canvas than a one line command called PAINT. This allows you to fill in any shape. That's right, any shape, with any available color.

CLOAD and LIST the next program.

Paint 75

Can you guess what each number on line sixty does? The numbers 135 and 125 are the coordinates for the point from which the computer will begin to paint using color number eight which is magenta. It will begin at point 135 comma 125 until it reaches a border.

RUN it and see for yourself.

But that's not all you can do with PAINT. When using SCREEN one, you are limited to painting up until you reach a border... any border. However when you use SCREEN two you can tell the computer to override any and all borders, to paint until it reaches a given color, and not to hang up its brush until it does.

CLOAD and LIST the program.

Paint2

You will notice that on line thirty we simply added another value to the paint command and away we go.

RUN it and see. If you don't like our colors, or dimensions, change them and see how artistic you are.

Computer coloring can be fun... no? Let's get into some architecturing now.

CLOAD and LIST the next program.

Paint Cubes 64

Line statments can only do so much for your program, that is, only one line or box at a time. By using the DRAW command, however, you can be your own boss, setting several lines of various lengths in any number of directions all in one statement! Take a look at line twenty.

The numbers 100 comma 100 are the coordinate of the starting point. We then moved the brush upward thirty units. That's right, the "U" determines upward motion, "D" stands for downward, "L" for left and "R" for right. Easy to remember isn't it. But what about the "BM" at the beginning of line twenty and the other lettrers thrown in along the way? O.K. We'll go thru it one at a time. The "M" in the beginning merely sets the DRAW command in motion and the "B" is necessary to insure that no unwanted lines will appear.

Fine. Take a pencial and paper and we'll walk thu line twenty one piece at a time. "BM" initiates the drawing on a clear slate. Do you have that piece of paper in front of yo yet? O.K. Pick a point at the center of the page. Assume that the point you picked is 100 units over and 100 units down. Now draw a line up, or north, 30 units from the starting point, then turn northeast and draw 15 units. You see, the "E" will draw a line at a 45 degree angle, while "F" draws at 135 degrees, "G" at 225 and "H" at 315 degrees.

Anyway, back to the program. The instruction "R30" draws 30 units to theright. "G15" draws at a line at 255 degrees and then "L30" draws 30 units to the left. Finish transferring the instructions of line twenty to your piece of paper and see what we've drawn. Can you guess? We've drawn open and closed cubes. What about you?

RUN the program to check it out.

Now, relist the cube program and we'll tell you some other nifty tricks.

Lines 20 thru 40 couldhave been written as one line. The same with line 90 and 100. Simply by adding the instruction "N" before a direction

instruction we can tell the computer to return to its previous position after drawing. For example:

"BM 100 comma 100 U30 R30 D30 L30" will draw a square.

On the other hand the command:

"BM 100 comma NU30 NR30 ND30 NL30" will draw intersecting lines at right angles across their middle. That's because the computer starts at position 100 comma 100 then goes up 30 units, then jumps back to position 100 comma 100 then goes right 30 units then jumps back to position 100 comma 100 etc.

Try your hand a rewriting those 5 line sof code into 2 lines. Just trace a path along the diagrams you made earlier. Remember, adding the letter "N" will jump you back to home base.

Don't worry if your canvas wasn't as bright as the computer's. Remember, you only had a pencil to work with. The computer has a paint brush.

Before going on, we should mention a few special features that can add to the draw statement.

CLOAD and LIST the next program and we'll illustrate them.

Draw + 87

Suppose you've drawn a square and then watned to draw a second one nearby. If you know exactly how far away you want it, you need not locate the two coordinates. For example, in line thirty of this program we have told the computer to draw a square that stands on one of its corners. The current or last draw position is 128 comma 96... it's a square remember. Line 40 adds a second square that rests firmly on its feet, which is located 25 units to the right and 25 units down from the most current draw position. Had we written "BM + plus 25 coma minus 25 the second square would have been drawn 25 over and 25 up. Can you figure out why?

O.K. Just to make sure that we've explained things well enough, RUN It and see for yourself.

Another thing. Let's say you've written your program, drawn your figures, and then decide things have gotten too crowded on your screen. You can scale down and for that mater scale up, any figure by simply inserting an "S" for scale, and a number.

CLOAD and LIST the next program

Scale 91

The "S4" instruction on line thirty leaves the picture unchanged, while the "S2" instruction on line 40 scales the picture to half its original size and if we used the "SB" instruction the picture would have doubled in size, and so on.

Now, RUN it and see for yourself.

In fact, we could have left out the "S4" instruction on line 30 and the computer would have assumed to draw in full scale. However, once a scale is set, you need to specify a second scale in order to change it. To see what we mean, delete "S4" in line 30 and RUN it again.

Why did it do that? Simple. The computer still had "S2" on its mind from your last run thru the program. That's why it helps to include the "S4".

There's more. If a letter "C" instruction followed by a number is inserted in a draw command the lines drawn can be in various colors. and a "B" instruction inserted in a draw command will draw an invisible or blank line. Useful, isn't it.

CLOAD and LIST the next program to learn about these new features.

Draw + C 96

This little program will draw a square with one side blank. The "B" in front of the down instruction will blank that segment and that segment only. Can you visualize it? RUN it to make sure.

If you're a little frustrated, stay calm. Try tis little exercise. Pick up your pencil and write down your name. Use block letters. Then, remembering the moves you made, write a program to draw your name on the graphics screen. If your first attempt leaves you wondering, try it again. When you're satisfied, just tune in again. We'll be here.

We're back. Imagine you see a friend down the block and are trying to catch up to say hello.

CLOAD and LIST the next programand

Get Put 100

Lines 20 through 70 represent your friend. Line 80 creates a rectangular array large enough to hold him, or her. Remember, we have explained what an array is in the program that stored a student's test scores and in the program that stored the information about a manuscript.

Line 90 is very crucial. It GETs the information that follows the word GET and places it in container "C". So on line 90 it will take the two

sets of points that are specified, which had they been drawn would have created a rectangle, and places this picture in "C". The next 5 lines simply PUT the contents of "C" at those locations on the screen. Ready?

RUN the program. See how easy it is to catch up. Then type LIST

Did you notice that we took a larger rectangular area than was necessary and put it in "C". Try cutting it down a little. Edit line 90 to read as follows:

GET, open parenthesis, 118, comma, 88, close parenthesis, hyphen, open parenthesis, 138, comma, 128, close parenthesis, comma, "C", comma, "PSET".

This time you won't encrouch on anyone else's space. Good fences make good neighbors. RUN the revised program and you'll see what we mean.

Had enough coloring for a while? Why not move over to the conservatory and we'll play you a tune.

CLOAD our next masterpiece and RUN it. After you RUN it it will list itself. Play 10 5

First off, you'll be glad to know that the PLAY command is not a graphics function. That is why line 10 doesn't have the instruction SCREEN but rather SOUND. When you have a chance, read about the SOUND command in the Reference Guide. There you'll be introduced to multiple channels, stereo, if you will.

But for now, let's just keep in mind the basics. PLAY is the name of the game. Line 70 asks the computer to play a set of five strings, each defined in the previous five lines.

Look at line 20. Can you guess what "T" stands for? That's right, tempo. You can determine the rhythm. The notes you can use range from "A" thru "G" and a number sign or plus sign next to the note means to play it sharp and minus sign means to play it flat. The "L" instruction sets the length of the note, while "R" stands for a rest period and "O" for any one one of 8 octaves. Simply add a number to each of these instructions to give it the value that you desire.

Tired? There's more, but that should be enough to get you thru your first few songs. Ready to play the saints again, or to create your own concerto? Go ahead and relax with some music for a while.

Are you ready to move from music to majic? We're going to walk you thru a couple of magical wonders. You might like to reread the section on Sprites in the chapter on Advanced Graphics in the tutorial text which came with the SV-318 and have it with as you examine our next few programs.

CLOAD and RUN the program. Be sure to press control-stop to stop the program and then press LIST

Sprite 169

Have you figured out what is going on? A figure is placed at the center of the screen. Well, it's actually eight figures, each one stacked on top of the other. Then in uick succession, the figures rotate outwards.

Look at the listing of the program. The second statement on line one simply defines variables A thru Z and integers. lines 10 and 20 set up 2 arrays, each with 8 smaller containers. Line 30 reads information from the data lines and assigns it to container "A" dollar sign. The first 8 lines of data provide the shape you wish to put on the screen. Line 40 then converts the data code into binary strinds which consists of ones and zeros as its characters. Then it sews each pieces of the shape's designer clothing onto its adjacent piece and stores the whole outfit in container "B" dollar sign. Line 60 then assigns each part of the designer clothing to each part of the SPRITE's body.

As we explained in the text, a sprite is a like a geni. You design its shape and command it with simple BASIC commands to make it do your bidding. This little program hints at the wonderful things you can do with sprites. let's continue with our explanation of this program.

Jump to line 300. The instruction RESTORE 200 tells the computer to read the data that begins on line 200. Line 310 reads two values a time, called "X" and "Y", adding these values to those already assigned to the "X" and "Y" arrays. Look again at line 10. Between these 2 lines, that is, lines 10 and 310, "X one will first eual 128, and "Y one will first eual 95. Then "X two will eual 129 while "Y" two remains at 95 and so forth. Try hard to understand these lines before going on.

Now look at line 330. This pulls all our information together. Remember the GET and PUT combination of commands we had a couple of programs ago

couple of programs ago? Here we have PUT SPRITE. PUT the SPRITE that is specified at the end of the line, which in our case is represented by container "I" because it holds sprites numbered one through eight, and places it on surface number "I" at position "XI", comma "YI", in color number 15. Let's review that line in plain english.

We took our shape that was created in the first half of the program and was stored in different sprite containers, and placed them in 8 different locations on the screen. The GOTO 300 command on tells the computer to resore or return to the beginning of the data at line 200 and away we go again.

Don't worry. It takes a little practice and testing to appreciate the ease and power of working with sprites. Your Sv-318 is one of the few microcomputerş available in the world that allow you to harness the power of sprites right in the BASIC language without having to play around with the computers memory using numbers that don't make much sense to the casual observer.

We almost forgot to tell you another secret. The sprites return to home even though this is nowhere indicated in the program because this is a trick of the screen. As the figures fall off to the right of the screen, they reappear again on the left. Exciting isn't it? Tricks of the hand, that's all.

Last but not least some real, we mean, real fun. Before you CLOAD and RUN the next program here are the instructions how to play with it. Use the joystick that is built into the keyboard to control the sprite in this program and see what happens. Use the space bar to clear the screen. When you've played enough, press control-stop and LIST it. O.K. Now CLOAD and RUN it.

Look at line 30. The INTERVAL instruction is an important and powerful one. Your computer has a internal clock that completes 60 machine cycles every second. The command "ON INTERVAL epual 2 GOTO 200" sets the computer to jump to line 200 every second machine cycle, that 30 times in every second. Got that? O.K. We'll come back to it in a second.

Line 30 tells the computer to wait until you input your starting point. Please note that in this program the INPUT command works differently than in examples that you've seen before. You're used to a line beginning with print command, followed by your message, a colon and the INPUT command and its container. Here we did it all in one step, without the help of the print command.

Lines 60 and 70 you are already familiar with, they read in the data and set up the sprite. Line 80 tells the clock to begin to tick, with the instruction, INTERVAL ON. Now remember, when the clock reaches 2 machine cycles it will jump to line 200. So let's jump to line 200.

Line 200 shuts the clock off. As the REM statement tells on line 10, this isn't necessary, because whenever a subroutine is entered the clock is automatically shut off. It is however necessary in order to properly time its return to the scene.

O.K. Moving along, the "D" container that is mentioned on line 200 was assigned on line 90 to hold the information that is sent by the joystick control. And line 100 says that if the space bar was pressed then clear the screen. Line 200 checks to see if the joystick was moved. If it was moved then it jumps to line 270. Whenever you push it in a given direction it will evaluate the push you gave it and set "I" and "J" in accordance with where you are on the screen. You see, we wouldn't want you to fall of the edge. When it figures out in which direction you have pushed the joystick it jumps back to line 210.

Line 210 adds the values that are stored in "I" and "J" to "X" and "Y" respectively. "X" and "Y" are the containers that hold the numbers that determine the direction of the sprite.

In other words, if you move the joystick then "I" and "J" will receive new values. These new values will then be passed to "X" and "Y", therby causing the sprite to move around the screen. You are using the joystick to control the sprite's movement. By moving the joystick a line is drawn as the sprite moves around the screen and control is returned to wherever you were when INTERVAL epualed two. Fortunately, the clock will only start ticking again on command, when control is returned to line 80.

We hope that you have kept up with us. If you have followed us to this point that makes us very happey. If you have come this far then there is nothing that can stop you from continuing to discover the power of the SV-318.

We have included two more programs for your enjoyment. We will not explain them. They are here for your entertainment. When you are ready to see them, type CLOAD and RUN. When you had enough of the next program then press control-stop and CLOAD and RUN the final program on this tape.

Time is the only thing that stands between you and the power that lies within your SV-318 computer. Feel confident that when you are ready you can continue to grow with the SV family of affortable and expandable personal computers.

SD-235T-UM

PRINTED IN HONG KONG